

SUMMARY REPORT – WINDOWS FORMS LAB (EXERCISE 1–10)

I. Introduction

Trong các bài lab này, em được làm quen với lập trình Windows Forms trên Visual Studio và cách tạo các ứng dụng giao diện đơn giản.

II. What I learned

Qua các bài từ 1 đến 10, em học được:

- Tạo project Windows Forms
- Sử dụng Label, Button, TextBox
- Xử lý sự kiện khi bấm nút
- Hiển thị dữ liệu bằng MessageBox
- Sử dụng DataGridView để hiển thị bảng dữ liệu
- Thêm, sửa, xóa dữ liệu trong DataGridView
- Sử dụng ComboBox, ListBox
- Sử dụng DateTimePicker và ProgressBar

III. Skills gained

Sau khi làm xong lab, em:

- Biết thiết kế giao diện cơ bản
- Biết xử lý sự kiện trong chương trình
- Biết làm việc với dữ liệu dạng bảng
- Hiểu cách hoạt động của Windows Forms

IV. Self-evaluation / Homework

Ban đầu em còn chưa quen với Visual Studio và cách dùng Toolbox, nhưng sau khi làm nhiều bài

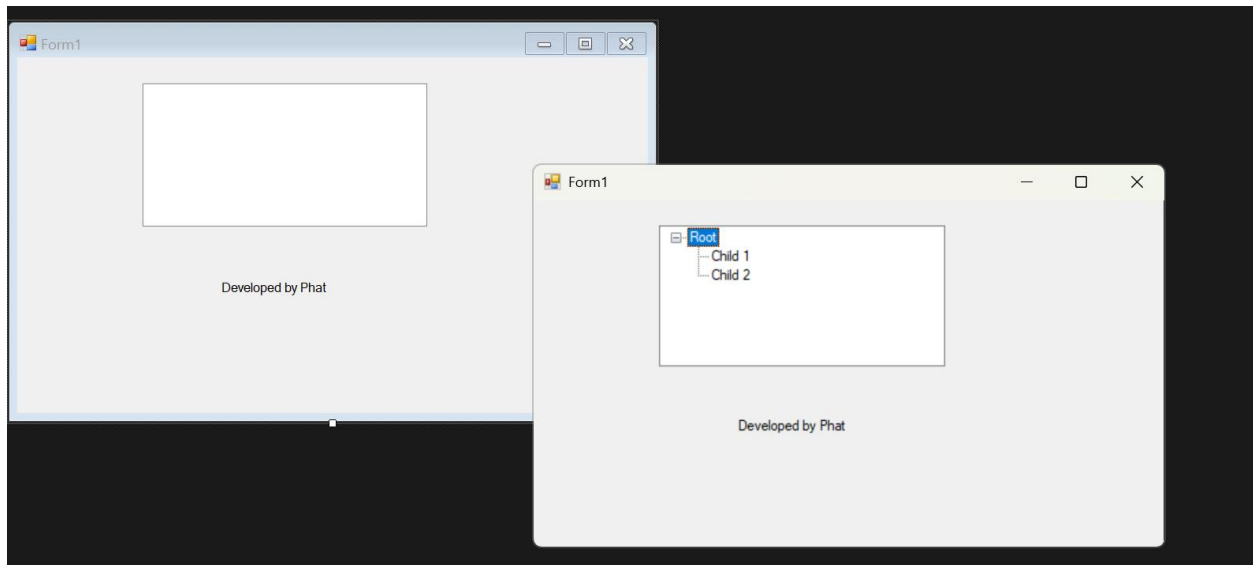
thì em đã hiểu rõ hơn.

Các bài lab giúp em nắm được kiến thức nền tảng để tiếp tục học các phần nâng cao hơn sau này.

Bài 11:

- Trong bài này, em sử dụng TreeView để hiển thị dữ liệu theo dạng phân cấp. Em tạo một project Windows Forms mới, sau đó kéo TreeView và Label vào Form.
- Trong sự kiện Form Load, em tạo một node gốc tên là “Root” và hai node con là “Child 1” và “Child 2”, rồi thêm chúng vào TreeView.
- Ngoài ra, em thêm một Label với nội dung “Developed by Phat” để hiển thị thông tin tác giả theo yêu cầu.
- Khi chạy chương trình, TreeView hiển thị đúng cấu trúc dạng cây với các node cha và con.

```
1  using System;
2  using System.Collections.Generic;
3  using System.ComponentModel;
4  using System.Data;
5  using System.Drawing;
6  using System.Linq;
7  using System.Text;
8  using System.Threading.Tasks;
9  using System.Windows.Forms;
10
11 namespace Bai11Lab1
12 {
13     3 references
14     public partial class Form1 : Form
15     {
16         1 reference
17         public Form1()
18         {
19             InitializeComponent();
20
21         1 reference
22         private void Form1_Load(object sender, EventArgs e)
23         {
24             TreeNode rootNode = new TreeNode("Root");
25             TreeNode childNode1 = new TreeNode("Child 1");
26             TreeNode childNode2 = new TreeNode("Child 2");
27
28             rootNode.Nodes.Add(childNode1);
29             rootNode.Nodes.Add(childNode2);
30
31             treeView1.Nodes.Add(rootNode);
32
33         1 reference
34         private void label1_Click(object sender, EventArgs e)
35         {
36
37         }
38     }
39 }
```



Bài 12:

- Trong bài này, em sử dụng **ListView** để hiển thị dữ liệu dạng danh sách theo chế độ **Details**.
- Em tạo project mới, kéo **ListView** và **Label** vào **Form**, sau đó đặt thuộc tính **View** của **ListView** là **Details**.
- Trong sự kiện **Form Load**, em thêm hai cột và hai dòng dữ liệu mẫu vào **ListView**.
- Ngoài ra, em thêm **Label** với nội dung “Developed by ...” theo yêu cầu.
- Khi chạy chương trình, **ListView** hiển thị dữ liệu dưới dạng bảng gồm các cột và dòng.

```
public partial class Form1 : Form
{
    1 reference
    public Form1()
    {
        InitializeComponent();
    }

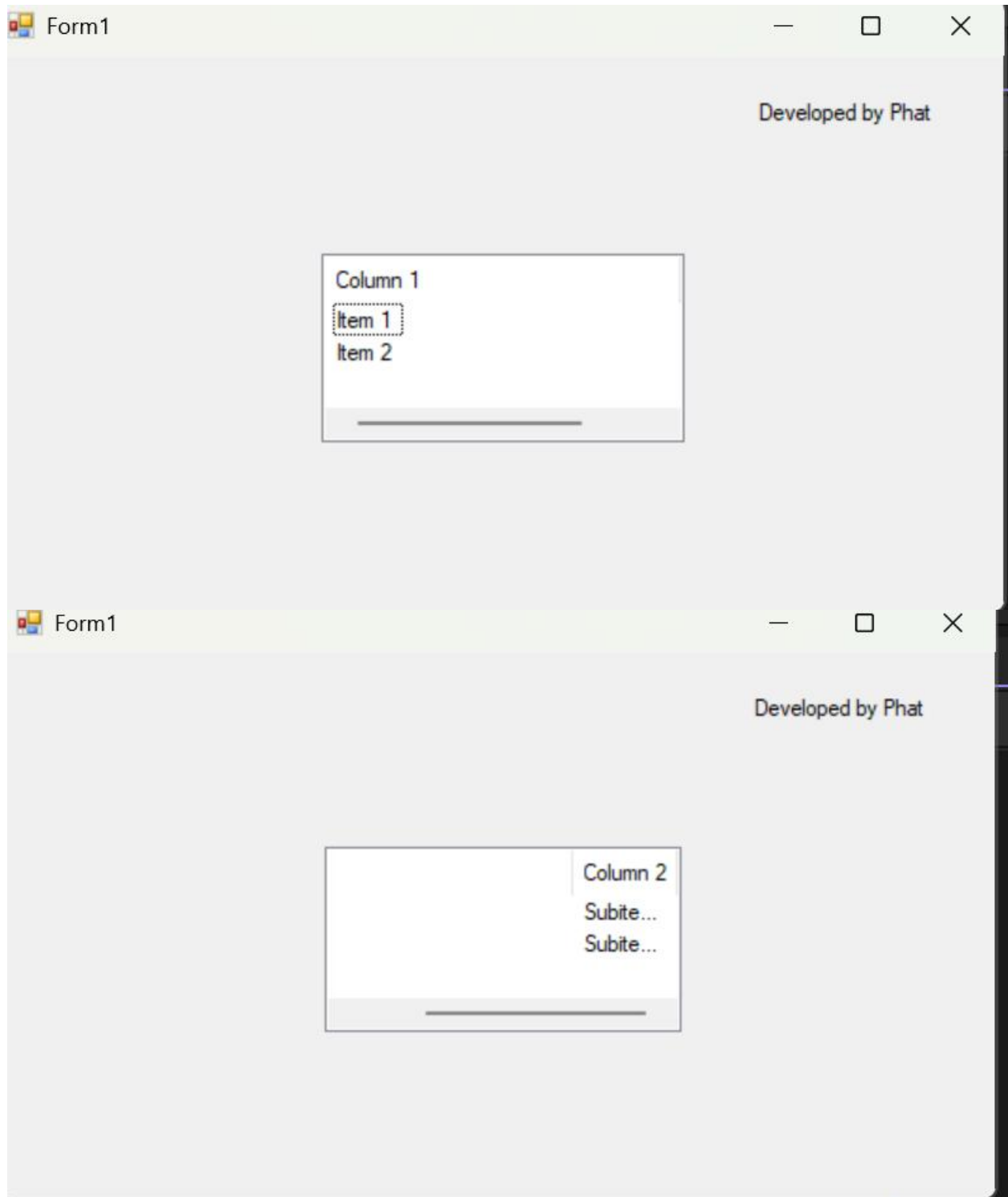
    1 reference
    private void Form1_Load(object sender, EventArgs e)
    {
        listView1.View = View.Details;

        listView1.Columns.Add("Column 1", -2, HorizontalAlignment.Left);
        listView1.Columns.Add("Column 2", -2, HorizontalAlignment.Left);

        ListViewItem item1 = new ListViewItem("Item 1");
        item1.SubItems.Add("Subitem 1");

        ListViewItem item2 = new ListViewItem("Item 2");
        item2.SubItems.Add("Subitem 2");

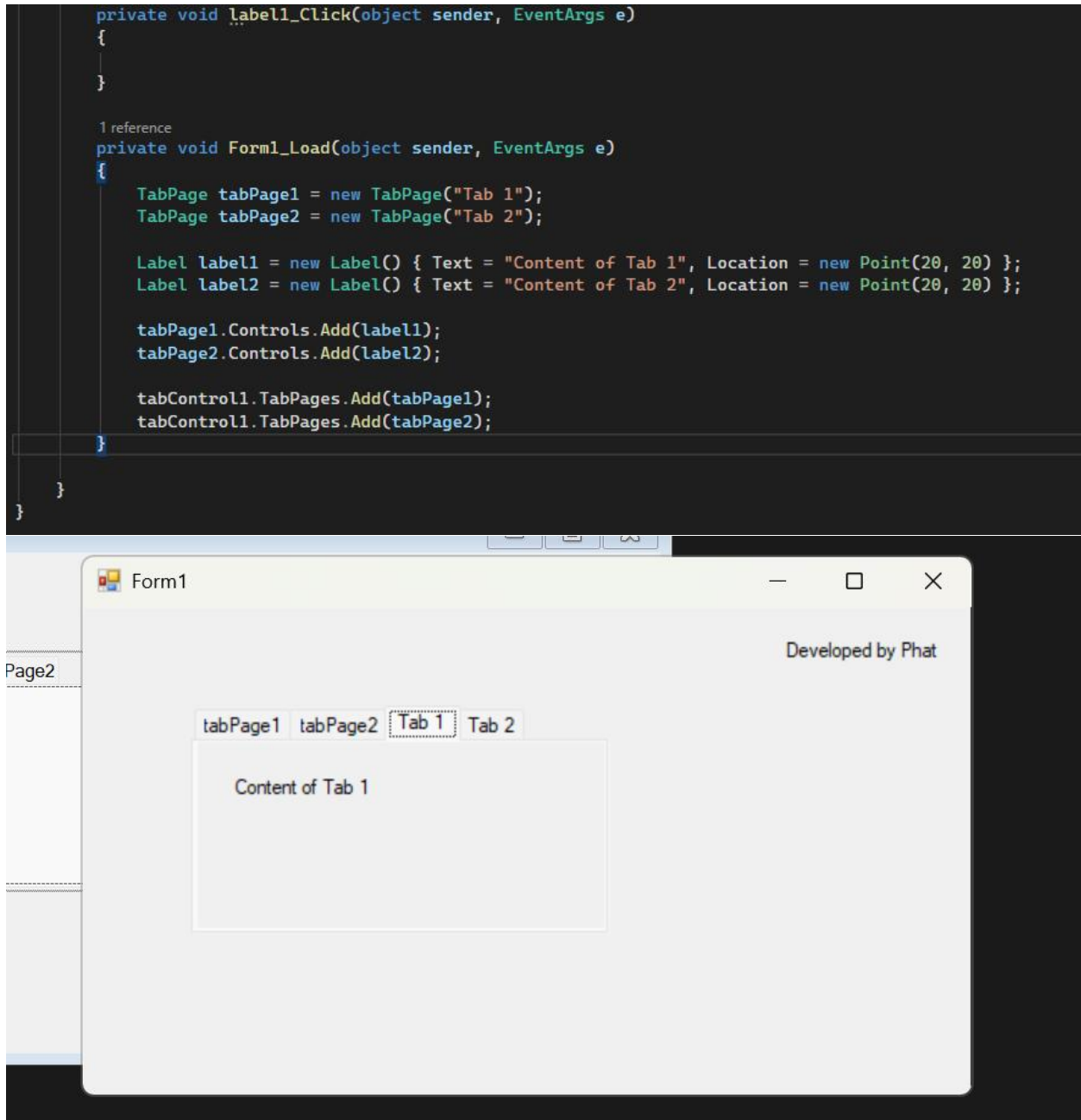
        listView1.Items.Add(item1);
        listView1.Items.Add(item2);
    }
}
```



Bài 13:

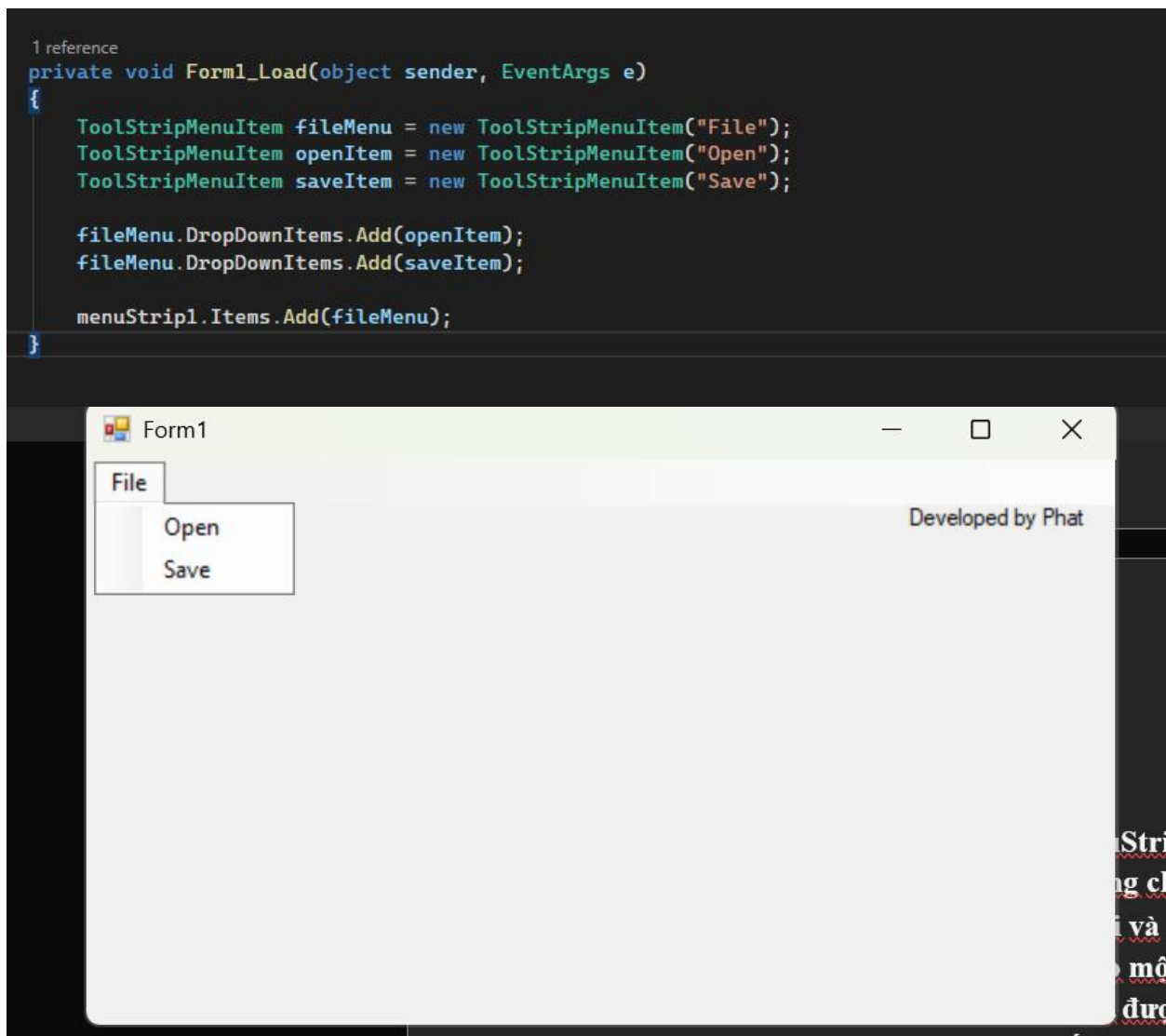
- Trong bài này, em sử dụng TabControl để tạo giao diện gồm nhiều tab.

- Em tạo project mới, kéo TabControl và Label vào Form. Trong sự kiện Form Load, em tạo hai tab là “Tab 1” và “Tab 2”, sau đó thêm Label vào mỗi tab để hiển thị nội dung.
- Ngoài ra, em thêm Label “Developed by ...” theo yêu cầu.
- Khi chạy chương trình, giao diện hiển thị các tab và có thể chuyển đổi giữa các tab để xem nội dung khác nhau.



Bài 14:

- Trong bài này, em sử dụng MenuStrip để tạo menu cho ứng dụng Windows Forms nhằm tổ chức các chức năng trong chương trình.
- Đầu tiên, em tạo một project mới và kéo MenuStrip từ Toolbox vào Form. Sau đó, trong sự kiện Form Load, em tạo một menu chính có tên là “File” và hai mục con là “Open” và “Save”. Các mục này được thêm vào menu chính bằng cách sử dụng DropDownItems.Add() và cuối cùng thêm menu vào MenuStrip.
- Khi chạy chương trình, menu “File” xuất hiện trên giao diện, khi bấm vào sẽ hiển thị các lựa chọn “Open” và “Save”, giúp người dùng dễ dàng thao tác với chương trình.



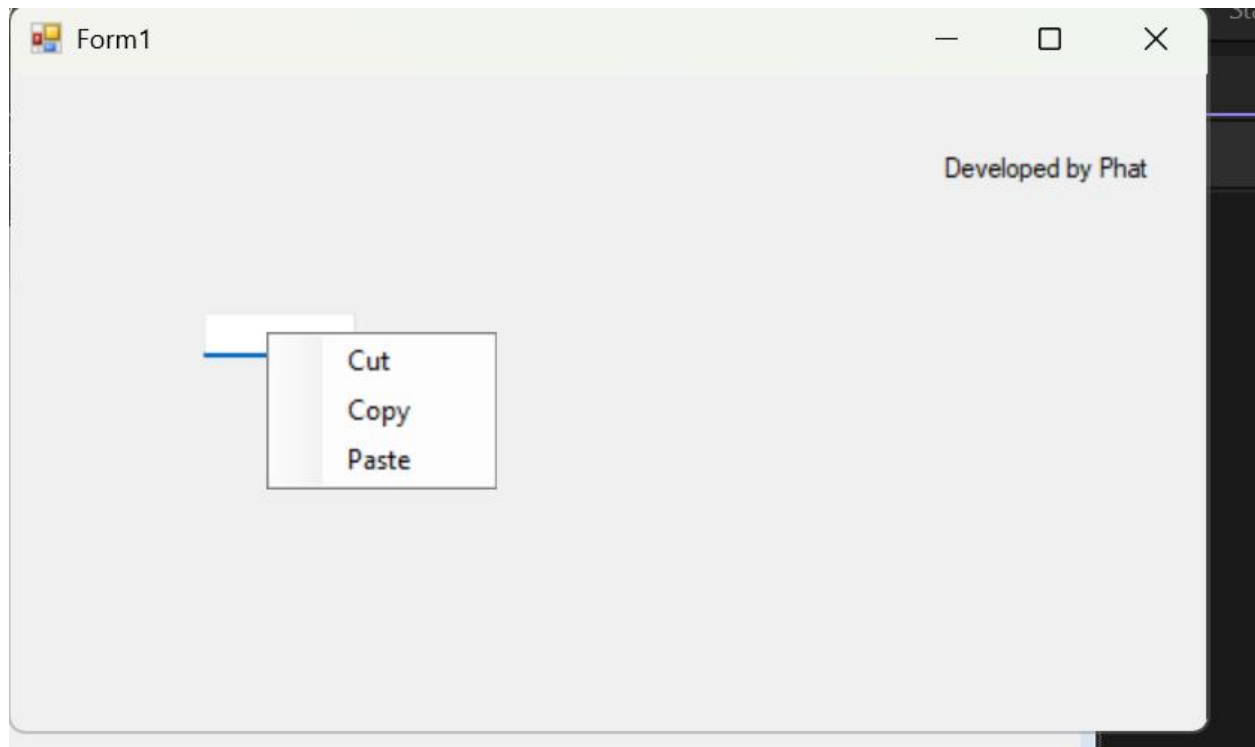
Bài 15:

- Trong bài này, em sử dụng **ContextMenuStrip** để tạo menu chuột phải cho một control trong Windows Forms, cụ thể là **TextBox**, giúp người dùng thao tác nhanh hơn khi nhập liệu.
- Đầu tiên, em tạo một project Windows Forms mới, sau đó kéo **ContextMenuStrip** và **TextBox** từ Toolbox vào Form. Tiếp theo, trong sự kiện **Form Load**, em tạo ba mục menu là “Cut”, “Copy” và “Paste” bằng đối tượng **ToolStripMenuItem**, rồi thêm các mục này vào **ContextMenuStrip** bằng phương thức **Items.Add()**.
- Sau khi tạo xong menu, em gán **ContextMenuStrip** cho **TextBox** thông qua thuộc tính **ContextMenuStrip** để menu chỉ xuất hiện khi người dùng click chuột phải vào **TextBox**.
- Khi chạy chương trình, người dùng có thể click chuột phải vào **TextBox** để mở menu và chọn các chức năng như cắt, sao chép hoặc dán văn bản, giúp việc nhập dữ liệu thuận tiện hơn.

```
1 reference
private void Form1_Load(object sender, EventArgs e)
{
    ToolStripMenuItem cutItem = new ToolStripMenuItem("Cut");
    ToolStripMenuItem copyItem = new ToolStripMenuItem("Copy");
    ToolStripMenuItem pasteItem = new ToolStripMenuItem("Paste");

    contextMenuStrip1.Items.Add(cutItem);
    contextMenuStrip1.Items.Add(copyItem);
    contextMenuStrip1.Items.Add(pasteItem);

    textBox1.ContextMenuStrip = contextMenuStrip1;
}
}
```

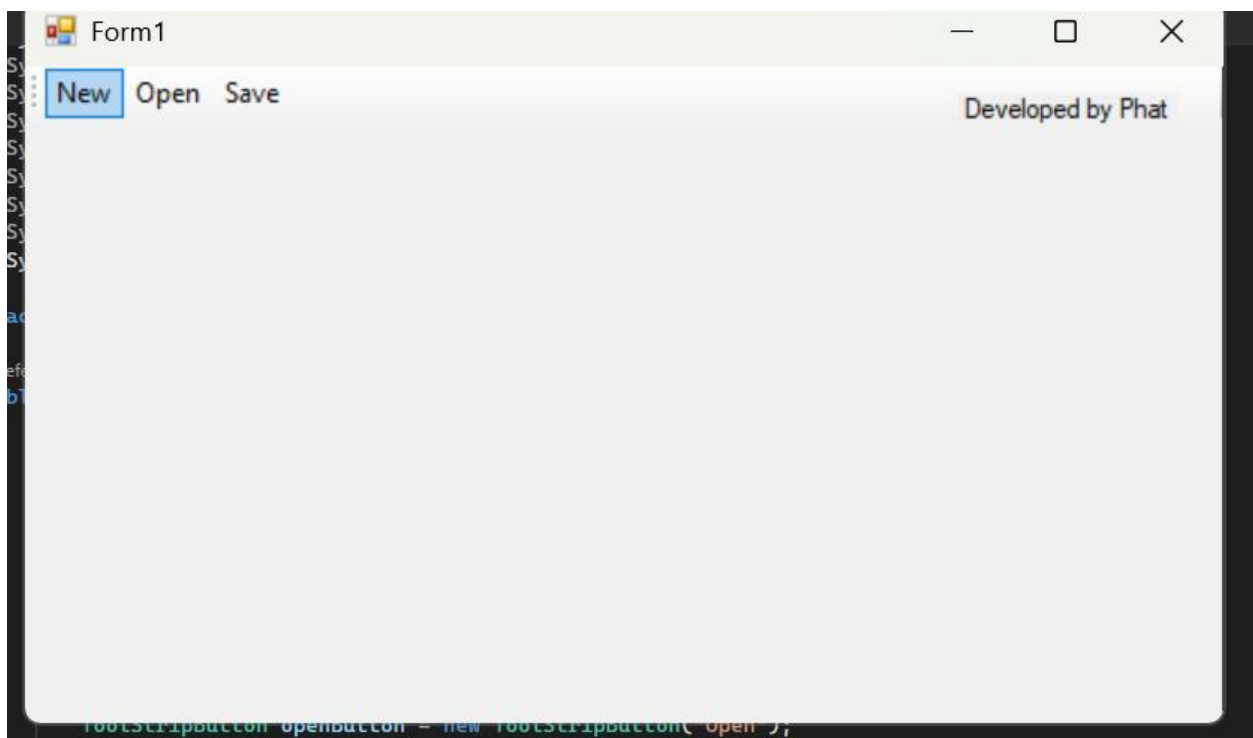



Bài 16:

- Trong bài này, em sử dụng ToolStrip để tạo một thanh công cụ (toolbar) cho ứng dụng Windows Forms, giúp người dùng truy cập nhanh các chức năng cơ bản.
- Em tạo project mới và thêm ToolStrip vào Form. Trong sự kiện Form Load, em tạo ba nút là "New", "Open" và "Save" bằng đối tượng ToolStripButton, sau đó thêm các nút này vào ToolStrip.
- Khi chạy chương trình, một thanh công cụ xuất hiện trên giao diện với các nút chức năng, giúp việc thao tác với chương trình trở nên thuận tiện hơn.

```
1 reference
private void Form1_Load(object sender, EventArgs e)
{
    ToolStripButton newButton = new ToolStripButton("New");
    ToolStripButton openButton = new ToolStripButton("Open");
    ToolStripButton saveButton = new ToolStripButton("Save");

    toolStrip1.Items.Add(newButton);
    toolStrip1.Items.Add(openButton);
    toolStrip1.Items.Add(saveButton);
}
}
```

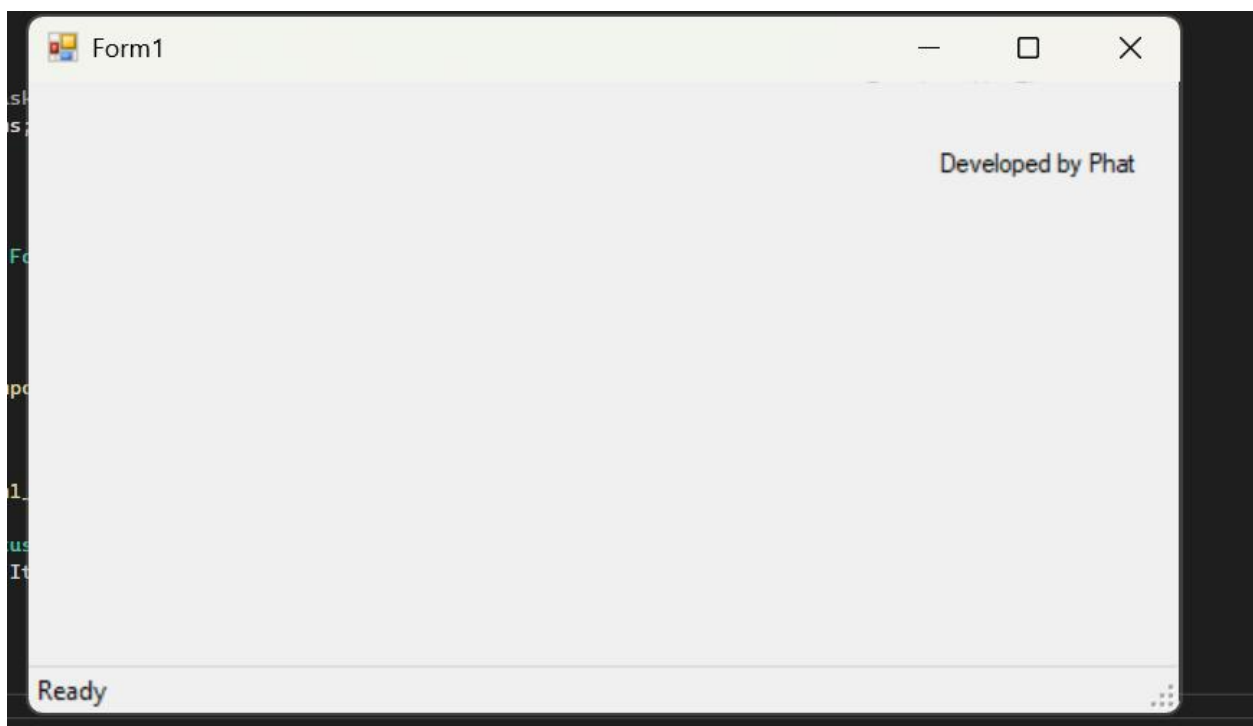


Bài 17:

- Trong bài này, em sử dụng *StatusStrip* để hiển thị thông tin trạng thái ở phía dưới cửa sổ chương trình.

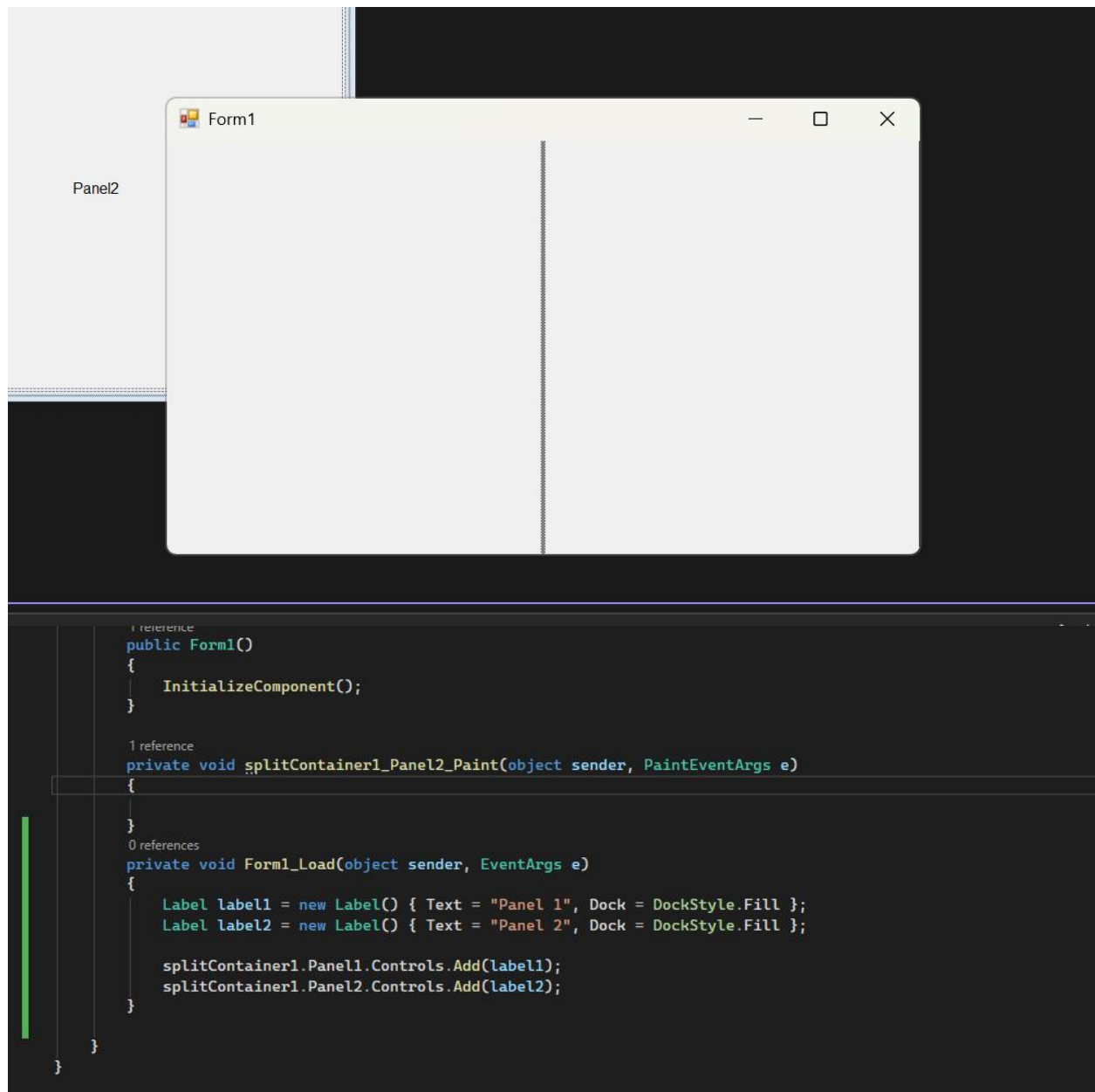
- Em tạo project mới và thêm control StatusStrip vào Form. Trong sự kiện Form Load, em tạo một ToolStripStatusLabel với nội dung “Ready” và thêm vào StatusStrip.
- Khi chạy chương trình, thanh trạng thái xuất hiện ở cuối Form, giúp hiển thị tình trạng hoạt động của ứng dụng.

```
}  
  
1 reference  
private void Form1_Load(object sender, EventArgs e)  
{  
    ToolStripStatusLabel statusLabel = new ToolStripStatusLabel("Ready");  
    statusStrip1.Items.Add(statusLabel);  
}  
}
```



Bài 18:

- Trong bài này, em sử dụng SplitContainer để chia Form thành hai vùng có thể thay đổi kích thước.
- Em tạo project mới và thêm SplitContainer vào Form. Trong sự kiện Form Load, em tạo hai Label là “Panel 1” và “Panel 2”, sau đó lần lượt thêm vào Panel1 và Panel2 của SplitContainer.
- Khi chạy chương trình, cửa sổ được chia thành hai phần và người dùng có thể kéo thanh phân cách để điều chỉnh kích thước của từng vùng.



Bài 19:

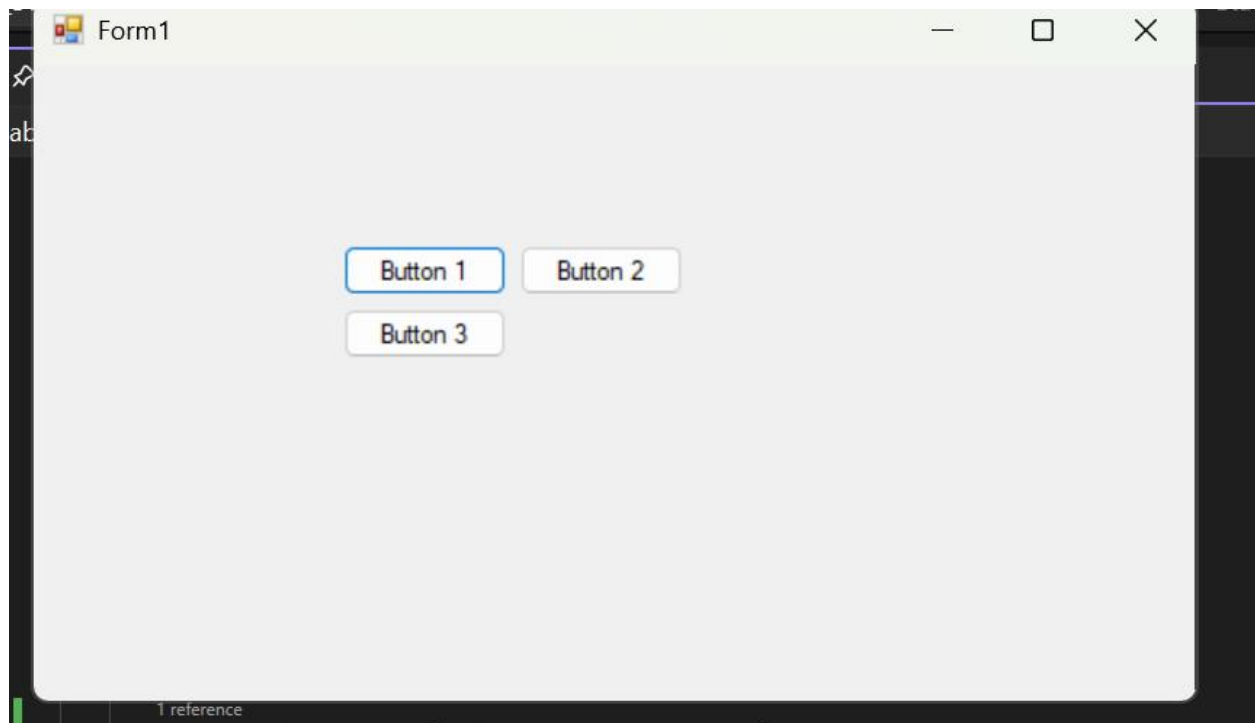
- Trong bài này, em sử dụng FlowLayoutPanel để sắp xếp các control theo dạng luồng tự động.
- Đầu tiên, em tạo một project Windows Forms mới và thêm control FlowLayoutPanel vào Form.
- Trong sự kiện Form Load, em tạo ba Button và thêm chúng vào FlowLayoutPanel thông qua thuộc tính Controls.Add().

- Khi chạy chương trình, các Button sẽ được sắp xếp tự động theo chiều ngang và tự xuống dòng khi không đủ không gian, giúp giao diện linh hoạt và dễ quản lý.

```
1 reference
public Form1()
{
    InitializeComponent();
}

1 reference
private void Form1_Load(object sender, EventArgs e)
{
    Button button1 = new Button() { Text = "Button 1" };
    Button button2 = new Button() { Text = "Button 2" };
    Button button3 = new Button() { Text = "Button 3" };

    flowLayoutPanel1.Controls.Add(button1);
    flowLayoutPanel1.Controls.Add(button2);
    flowLayoutPanel1.Controls.Add(button3);
}
}
```



Bài 20:

- Trong bài này, em sử dụng `TableLayoutPanel` để sắp xếp các control theo dạng bảng gồm các hàng và cột.
- Em tạo một project Windows Forms mới và thêm control `TableLayoutPanel` vào Form. Sau đó thiết lập số hàng và số cột là 2×2 , mỗi ô chiếm 50% kích thước.
- Trong sự kiện `Form Load`, em tạo 4 Button và đặt chúng vào từng ô tương ứng trong bảng bằng phương thức `Controls.Add()`.
- Khi chạy chương trình, các Button được hiển thị gọn gàng theo dạng lưới 2×2 và tự động thay đổi kích thước khi form thay đổi.

1 reference

```
private void Form1_Load(object sender, EventArgs e)
{
    tableLayoutPanel1.ColumnCount = 2;
    tableLayoutPanel1.RowCount = 2;

    tableLayoutPanel1.ColumnStyles.Add(new ColumnStyle(SizeType.Percent, 50F));
    tableLayoutPanel1.ColumnStyles.Add(new ColumnStyle(SizeType.Percent, 50F));

    tableLayoutPanel1.RowStyles.Add(new RowStyle(SizeType.Percent, 50F));
    tableLayoutPanel1.RowStyles.Add(new RowStyle(SizeType.Percent, 50F));

    Button button1 = new Button() { Text = "Button 1" };
    Button button2 = new Button() { Text = "Button 2" };
    Button button3 = new Button() { Text = "Button 3" };
    Button button4 = new Button() { Text = "Button 4" };

    tableLayoutPanel1.Controls.Add(button1, 0, 0);
    tableLayoutPanel1.Controls.Add(button2, 1, 0);
    tableLayoutPanel1.Controls.Add(button3, 0, 1);
    tableLayoutPanel1.Controls.Add(button4, 1, 1);
}
```

